S2

ACTIVIDAD PARCIAL 2

FUNDAMENTOS DE INTELIGENCIA ARTIFICIAL

Problemas y espacios de búsqueda

Sección 2 · 2.1 Problemas de búsqueda y 2.2 Espacios de búsqueda

Modelado de dos problemas reales como problemas de búsqueda y generación de su espacio de estados con NetworkX.

ALUMNO	Erandy Naomy Rosas Toriz
GRUPO	171 · Cuatrimestre 7
DOCENTE	Mtro. Javier García Lara
PERIODO	12 al 26 de octubre de 2026
ENTREGA	18 de octubre de 2026 · Individual
REPOSITORIO	FIA_G171_Rosas/S2_busqueda

CONTENIDO

1 Planteamiento
2 Formulación
3 Datos y procedimiento
4 Solución y resultados
5 Análisis de complejidad
6 Conclusiones
7 Declaración de uso de IA
Referencias y anexos

UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

Ing. en Tecnologías de la Información e Innovación Digital

1 Planteamiento del problema

Para que un programa pueda resolver un problema, primero hay que **representarlo** de forma que un algoritmo lo explore. En la IA clásica esto se hace como un **problema de búsqueda**: un agente parte de un *estado inicial*, aplica *acciones* y busca llegar a un estado *meta* con el menor costo posible (Russell & Norvig, 2020). Al conjunto de todos los estados alcanzables se le llama **espacio de estados** y se puede ver como un grafo: los nodos son estados y las aristas son acciones.

Esta actividad modela **dos problemas reales**, genera su espacio de estados como grafo dirigido y ponderado con NetworkX, calcula la solución óptima y analiza qué tan rápido crece el espacio al aumentar el tamaño del problema.

PREGUNTA GUÍA
¿Cómo se formula cada problema (estado, acciones, costo y meta), cuántos estados genera y qué tan costoso sería explorarlo?

Objetivos

- Formular dos problemas reales con los seis elementos de un problema de búsqueda.
- Programar en Python un **generador de estados** y el **grafo** del espacio de búsqueda con NetworkX.
- Encontrar la solución óptima, comprobarla por otro método y analizar la complejidad en tiempo y espacio.

Problemas seleccionados

	Problema 1 · Robot de almacén	Problema 2 · Reparto con 4 clientes
Situación real	Un robot de bodega o papelería lleva material desde la entrada hasta la zona de entrega esquivando estantes y pasillos ocupados.	Un repartidor sale del depósito, visita 4 clientes una sola vez y regresa.
Tipo de espacio	Cuadrícula: el espacio es pequeño y crece de forma lineal con el número de celdas.	Combinatorio: el espacio crece de forma exponencial con el número de clientes (tipo TSP).
Por qué se eligió	Es el modelo más simple de planeación de rutas y se relaciona con el laberinto de la S4.	Es el mismo tipo de problema del caso UPS ORION (S1) y prepara la optimización de la S3.

Metodología

Cada problema se formuló a mano y se implementó como una clase de Python con el mismo conjunto de métodos. Un único generador recorre el espacio en anchura y construye el grafo. La solución óptima se obtiene con el algoritmo de Dijkstra sobre ese grafo y, en el problema 2, se verifica probando las 24 rutas posibles por fuerza bruta. Por último se calculan métricas y se dibujan los diagramas.

2 Formulación de los problemas

2.1 Elementos de un problema de búsqueda

Elemento	Qué define
Estado	Descripción completa de la situación en un momento dado.
Estado inicial	Punto de partida del agente.
Acciones	Lo que el agente puede hacer en cada estado.
Modelo de transición	Estado al que se llega al aplicar una acción en un estado.
Costo de paso	Cuánto cuesta cada acción (distancia, tiempo, 1 por paso, etc.).
Prueba de meta	Condición que indica que el problema quedó resuelto.

2.2 Problema 1 · Robot de almacén

Elemento	Formulación
Estado	(fila, columna) de la celda donde está el robot.
Estado inicial	(0, 0), celda **S**.
Acciones	Arriba, Abajo, Izquierda, Derecha, si la celda destino está dentro del mapa y no es un estante (#).
Transición	resultado((f, c), a) = (f + Δf, c + Δc), con el desplazamiento de la acción a.
Costo de paso	1 por celda normal; **3** al entrar a una zona de tráfico lento (T).
Meta	Estado (4, 4), celda **M**.
Tamaño del espacio	21 estados alcanzables (25 celdas − 4 estantes).

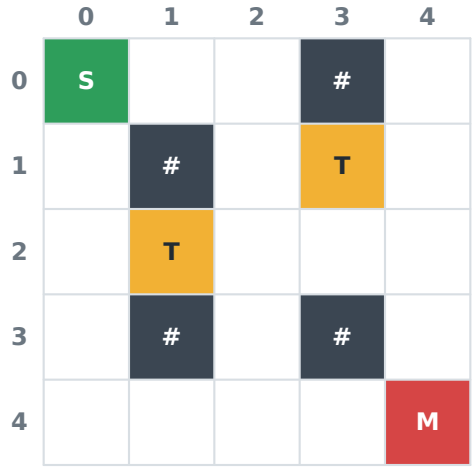
2.3 Problema 2 · Reparto con 4 clientes

Elemento	Formulación
Estado	(ubicación, visitados): ubicación ∈ {Depósito, A, B, C, D} y visitados es el conjunto de puntos ya atendidos.
Estado inicial	(Depósito, {Depósito}).
Acciones	Ir a un cliente aún no visitado; si ya se visitaron los cuatro, volver al depósito.
Transición	resultado((u, V), x) = (x, V ∪ {x}).
Costo de paso	Distancia en km entre la ubicación actual u y el destino x.
Meta	Ubicación = Depósito y visitados = {Depósito, A, B, C, D}.
Tamaño del espacio	$2 + n \cdot 2^{n-1} = 34$ estados para n = 4 clientes.

¿POR QUÉ EL ESTADO INCLUYE «VISITADOS»?
Si el estado fuera solo la ubicación, el agente no sabría qué clientes ya atendió y podría repetirlos o regresar sin terminar. Guardar el conjunto de visitados hace que el estado resuma todo lo necesario para decidir lo que sigue.

3 Datos de entrada y procedimiento

3.1 Datos del problema 1: mapa del almacén



Símbolo	Significado
S	Inicio del robot (0, 0)
M	Meta (4, 4)
#	Estante: celda bloqueada
T	Tráfico lento: costo 3
vacía	Pasillo libre: costo 1

Figura 1. Mapa de entrada del problema 1 (filas y columnas numeradas desde 0).

3.2 Datos del problema 2: distancias entre puntos (km)

	Dep	A	B	C	D
Dep	—	4	7	6	9
A	4	—	3	5	8
B	7	3	—	4	5
C	6	5	4	—	3
D	9	8	5	3	—

Tabla 1. Matriz de distancias simétrica. Son valores de ejemplo definidos para la actividad, no mediciones reales; en el código se pueden reemplazar por distancias reales.

3.3 Procedimiento

- **Definir** el problema como clase (`ProblemaBusqueda`) con estado inicial, acciones, resultado, costo y meta.
- **Generar el espacio** con `generar_grafo()`: recorrido en anchura que agrega cada estado nuevo como nodo y cada acción como arista con su costo.
- **Resolver** con `nx.dijkstra_path()` sobre el grafo para obtener la ruta de menor costo.
- **Medir** nodos, aristas, factor de ramificación y profundidad con `analizar()`.
- **Verificar** el problema 2 con `verificar_por_fuerza_bruta()` (24 rutas) y **dibujar** los diagramas.

```
generar_grafo(problema):
    G <- grafo dirigido vacio
    inicio <- problema.estado_inicial()
    cola <- [inicio]
    mientras cola no este vacia:
        s <- sacar el primer estado de cola
        para cada accion a en problema.acciones(s):
            n <- problema.resultado(s, a)
            si n no esta en G: agregar n a G y a cola
            agregar arista (s -> n) con peso costo(s, a, n)
    devolver G
```

Pseudocódigo del generador de estados (el código completo está en el Anexo A).

4 Solución y resultados

4.1 Problema 1 · Robot de almacén

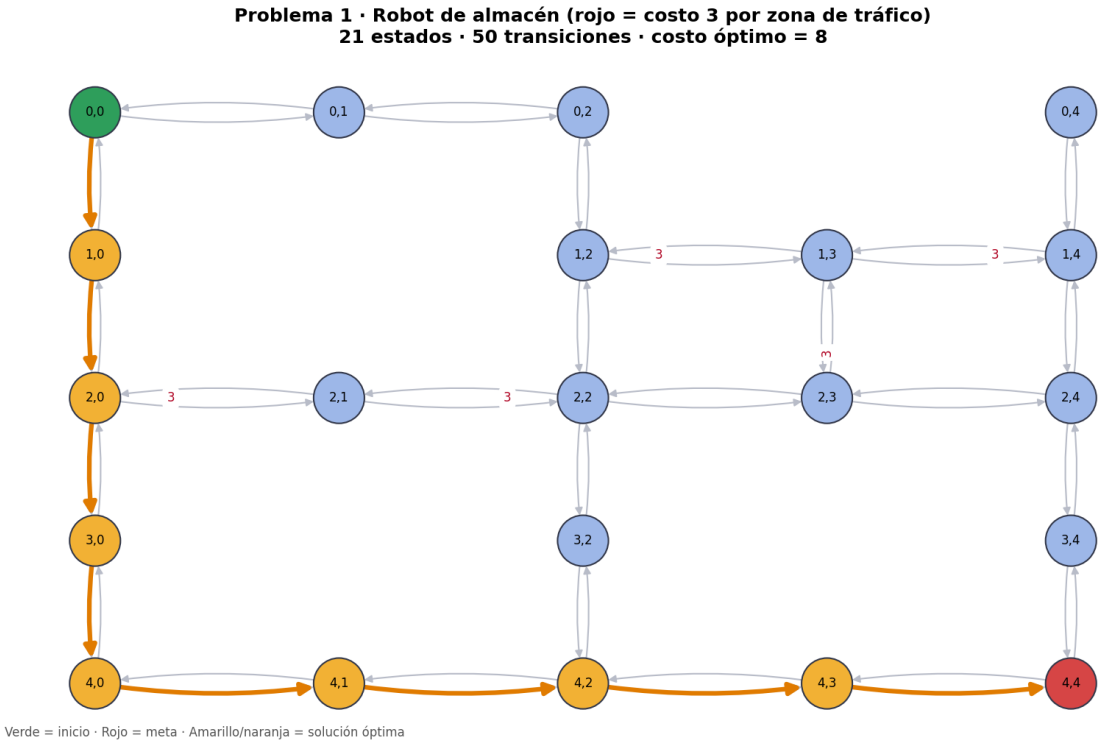


Figura 2. Espacio de estados del robot de almacén. Verde = inicio, rojo = meta, naranja = ruta óptima. Las aristas con «3» son entradas a zonas de tráfico lento.

Métrica	Resultado
Estados (nodos)	21
Transiciones (aristas)	50
Factor de ramificación	promedio ≈ 2.38; máximo b = 4
Profundidad de la solución	d = 8 acciones
Costo óptimo (Dijkstra)	8
Ruta óptima	(0,0) → (1,0) → (2,0) → (3,0) → (4,0) → (4,1) → (4,2) → (4,3) → (4,4)

EL COSTO IMPORTA
Existe otra ruta de igual longitud (8 pasos) que cruza la zona de tráfico lento en (1, 3). Su costo sería **10** en lugar de 8, por eso Dijkstra la descarta aunque tenga el mismo número de pasos.

4.2 Problema 2 · Reparto con 4 clientes

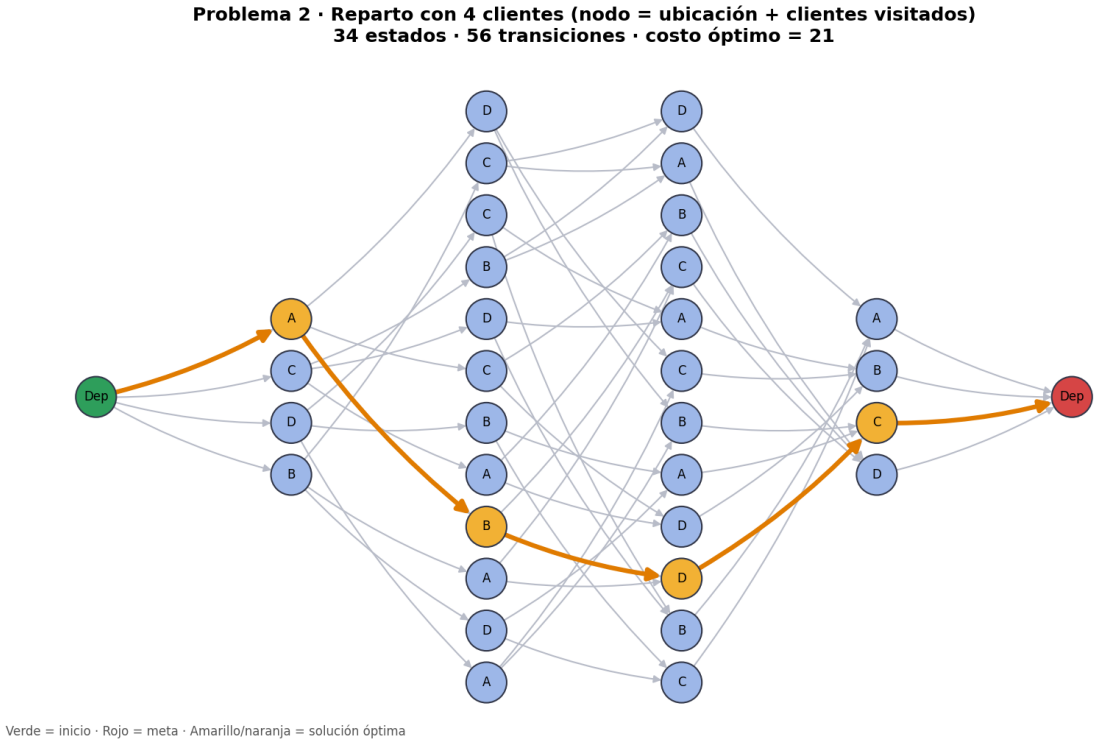


Figura 3. Espacio de estados del reparto. Cada columna agrupa los estados con el mismo número de puntos visitados; la ruta óptima va en naranja.

Métrica	Resultado
Estados (nodos)	34
Transiciones (aristas)	56
Factor de ramificación	promedio ≈ 1.65; máximo b = 4
Profundidad de la solución	d = 5 acciones
Costo óptimo (Dijkstra)	21 km
Ruta óptima	Dep → A → B → D → C → Dep

Verificación por fuerza bruta. Se calcularon las 24 rutas posibles; la mejor coincide con la que encontró Dijkstra sobre el grafo.

#	Ruta	Costo (km)
1	Dep → A → B → D → C → Dep	21
2	Dep → C → D → B → A → Dep	21
3	Dep → A → B → C → D → Dep	23
4	Dep → D → C → B → A → Dep	23
5	Dep → A → C → D → B → Dep	24
24	Dep → D → A → C → B → Dep (peor)	33

Tabla 2. Las cinco mejores rutas y la peor de las 24 posibles.

4.3 Comparación de los dos problemas

	Robot de almacén	Reparto (n = 4)
Estados / aristas	21 / 50	34 / 56
Profundidad d	8	5
Costo óptimo	8	21 km
Crecimiento del espacio	Lineal: depende del número de celdas.	Exponencial: depende del número de clientes.
Dificultad al escalar	Baja: un mapa 100×100 tendría unas 10,000 celdas.	Alta: con 20 clientes ya hay más de 10 millones de estados.

5 Análisis de complejidad

Se usan tres parámetros: **b**, el factor de ramificación (acciones por estado); **d**, la profundidad de la solución; y **V** y **E**, el número de estados y de transiciones del grafo.

Estrategia	Tiempo	Espacio	Comentario
Anchura sobre árbol (sin detectar repetidos)	$O(b^d)$	$O(b^d)$	Re-explora los mismos estados muchas veces.
Anchura sobre grafo (con conjunto de visitados)	O(V + E)	O(V)	Cada estado se expande una sola vez.
Dijkstra con cola de prioridad	O((V + E) log V)	O(V)	Obtiene el camino de menor costo.

5.1 Problema 1

Con b máximo = 4 y d = 8, la cota en árbol es 4^8 = 65,536 nodos. Al detectar estados repetidos solo se necesitan V + E = 71 pasos. El espacio es polinomial en el tamaño del mapa: a lo más una celda por estado, es decir O(filas × columnas).

5.2 Problema 2

Con n clientes hay **n!** rutas posibles, pero el grafo solo tiene **$2 + n \cdot 2^{n-1}$** estados distintos, porque muchas rutas llegan al mismo par (ubicación, visitados) y se fusionan en un solo nodo. Esta idea es la base de la programación dinámica de Held y Karp (1962). Aun así, el crecimiento sigue siendo exponencial.

n clientes	Rutas posibles (n!)	Estados del grafo
4	24	34
5	120	82
10	3,628,800	5,122
15	1,307,674,368,000	245,762
20	2,432,902,008,176,640,000	10,485,762

Tabla 3. Crecimiento del espacio de búsqueda del problema 2 según el número de clientes.

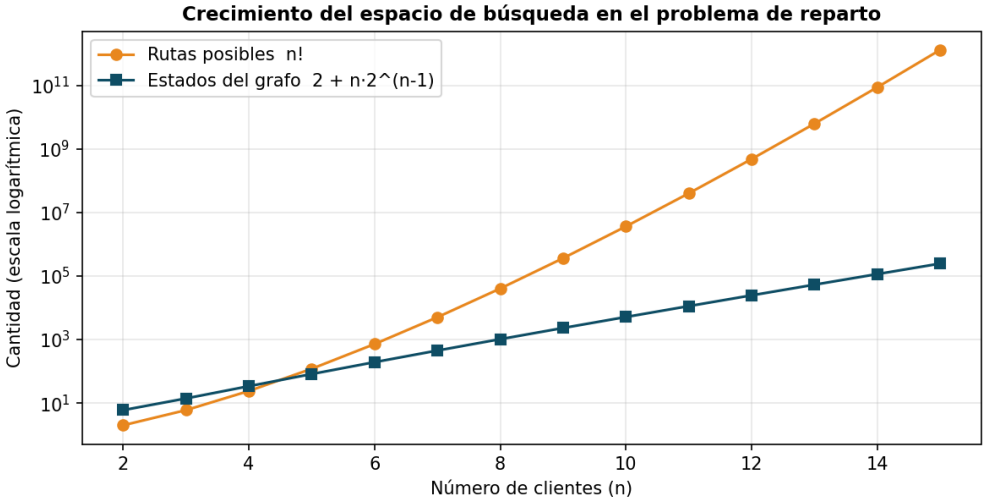


Figura 4. Rutas posibles frente a estados del grafo (escala logarítmica).

CONCLUSIÓN DEL ANÁLISIS
Para n = 4 basta con explorar todo el espacio, pero con 20 clientes ya hay más de 10 millones de estados y más de 2×10^{18} rutas. Por eso en la S3 se usan métodos de búsqueda local (Hill Climbing y Simulated Annealing) que no exploran todo el espacio.

6 Conclusiones reflexivas

- **Formular bien es la mitad del trabajo.** Elegir qué guarda el estado (por ejemplo, los clientes visitados) decide si el problema se puede resolver y cuántos estados tendrá.
- **El grafo hace visible el espacio de búsqueda.** Permite contar estados, medir la ramificación y ver las rutas alternativas, algo que no se aprecia solo con el código.
- **El costo cambia la solución.** En el almacén dos rutas de 8 pasos tienen costos distintos (8 y 10); un algoritmo que solo cuente pasos podría elegir mal.
- **La complejidad limita los métodos exactos.** Probar todas las rutas funciona con 4 clientes y deja de ser viable con unas pocas decenas, lo que justifica las heurísticas.
- **Limitaciones.** Los mapas y distancias son de ejemplo y de tamaño pequeño. En un caso real habría que usar distancias de un servicio de mapas, considerar horarios y capacidades, y validar el modelo con datos de la operación.

7 Declaración de uso de inteligencia artificial

Herramienta	Claude (Anthropic), modelo Claude Sonnet 5.5, con ejecución de código Python.
Prompt (resumen)	Se entregó la hoja de la Actividad Parcial 2 y se pidió el código y el PDF de la sección 2 (2.1 Problemas de búsqueda y 2.2 Espacios de búsqueda).
Uso	Diseño de la formulación de los dos problemas, escritura del código (clases, generador de estados, grafo con NetworkX, gráficas), redacción del borrador y maquetación del PDF (reportlab).
Verificación	El código se ejecutó completo. Los resultados (21 y 34 estados, 50 y 56 transiciones, costos óptimos 8 y 21) se contaron con NetworkX y la ruta del reparto se comprobó probando las 24 rutas. Las distancias del reparto son valores de ejemplo. Las referencias deben cotejarse con las fuentes originales antes de entregar.

R Referencias (APA 7.ª ed.)

Bohm, R. A. (2021). *The fundamentals of search algorithms*. Nova Science.

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik, 1*(1), 269-271. https://doi.org/10.1007/BF01386390

Hagberg, A. A., Schult, D. A., & Swart, P. J. (2008). Exploring network structure, dynamics, and function using NetworkX. En G. Varoquaux, T. Vaught & J. Millman (Eds.), *Proceedings of the 7th Python in Science Conference* (pp. 11-15).

Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics, 10*(1), 196-210.

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90-95. https://doi.org/10.1109/MCSE.2007.55

Russell, S., & Norvig, P. (2020). *Inteligencia Artificial: Un enfoque moderno* (4.ª ed.). Pearson.

A Anexo A. Código Python

Repositorio: FIA_G171_Rosas/S2_busqueda · archivo s2_problemas_busqueda.py

```
"""S2 · Problemas de búsqueda y espacios de búsqueda (2.1 y 2.2)

Modela 2 problemas reales como problemas de búsqueda (estado, acciones, costo, meta),
genera el espacio de estados como grafo dirigido y ponderado con NetworkX, dibuja el
diagrama y calcula métricas de complejidad.

  Problema 1: Robot de almacén (papelería/bodega) en una cuadrícula con obstáculos
              y zonas de tráfico lento.
  Problema 2: Repartidor que visita 4 clientes desde un depósito y regresa (tipo TSP,
              el mismo tipo de problema del caso UPS ORION de la S1).

Ejecutar: python s2_problemas_busqueda.py
Requiere: pip install networkx matplotlib
Genera:   figuras/grafo_almacen.png, figuras/grafo_reparto.png y figuras/crecimiento_estados.png
"""
import itertools
import math
import os
from collections import deque

import matplotlib
matplotlib.use("Agg")  # genera PNG sin abrir ventana
import matplotlib.pyplot as plt
import networkx as nx

AQUI = os.path.dirname(os.path.abspath(__file__))
FIG = os.path.join(AQUI, "figuras")
os.makedirs(FIG, exist_ok=True)


# ============================================================================
# 1. Formulación general de un problema de búsqueda
# ============================================================================
class ProblemaBusqueda:
    """Un problema se define con: estado inicial, acciones, modelo de transición,
    costo de paso y prueba de meta (Russell & Norvig, cap. 3)."""
    nombre = "Problema"

    def estado_inicial(self):
        raise NotImplementedError

    def acciones(self, estado):
        """Lista de acciones aplicables en 'estado'."""
        raise NotImplementedError

    def resultado(self, estado, accion):
        """Estado que resulta de aplicar 'accion' en 'estado'."""
        raise NotImplementedError

    def costo(self, estado, accion, nuevo):
        """Costo de paso (por defecto 1)."""
        return 1

    def es_meta(self, estado):
        raise NotImplementedError


# ============================================================================
# 2. Generador de estados + grafo (sirve para cualquier problema)
# ============================================================================
def generar_grafo(problema):
    """Explora todo el espacio alcanzable (recorrido en anchura) y lo guarda en un
    grafo dirigido: nodo = estado, arista = acción, peso = costo de paso."""
    G = nx.DiGraph()
    inicio = problema.estado_inicial()
    G.add_node(inicio)
    cola = deque([inicio])
    while cola:
        s = cola.popleft()
        for a in problema.acciones(s):
            n = problema.resultado(s, a)
            if n not in G:          # estado nuevo -> se agrega y se explorará
```

```
                G.add_node(n)
                cola.append(n)
            G.add_edge(s, n, weight=problema.costo(s, a, n), accion=a)
    return G


# ============================================================================
# 3. Problema 1: robot de almacén en cuadrícula
# ============================================================================
class RobotAlmacen(ProblemaBusqueda):
    """Estado: (fila, columna). Acciones: Arriba, Abajo, Izquierda, Derecha.
    Costo: 1 por celda normal, 3 por celda 'T' (tráfico lento / pasillo ocupado).
    Meta: llegar a 'M'. '#' = estante (obstáculo)."""
    nombre = "Robot de almacén"
    MAPA = [
        "S..#.",
        ".#.T.",
        ".T...",
        ".#.#.",
        "....M",
    ]
    MOV = {"Arriba": (-1, 0), "Abajo": (1, 0), "Izquierda": (0, -1), "Derecha": (0, 1)}

    def __init__(self):
        self.filas, self.cols = len(self.MAPA), len(self.MAPA[0])
        for r, fila in enumerate(self.MAPA):
            for c, ch in enumerate(fila):
                if ch == "S":
                    self.inicio = (r, c)
                elif ch == "M":
                    self.meta = (r, c)

    def estado_inicial(self):
        return self.inicio

    def acciones(self, estado):
        r, c = estado
        libres = []
        for nombre, (dr, dc) in self.MOV.items():
            nr, nc = r + dr, c + dc
            if 0 <= nr < self.filas and 0 <= nc < self.cols and self.MAPA[nr][nc] != "#":
                libres.append(nombre)
        return libres

    def resultado(self, estado, accion):
        dr, dc = self.MOV[accion]
        return (estado[0] + dr, estado[1] + dc)

    def costo(self, estado, accion, nuevo):
        return 3 if self.MAPA[nuevo[0]][nuevo[1]] == "T" else 1

    def es_meta(self, estado):
        return estado == self.meta


# ============================================================================
# 4. Problema 2: repartidor con 4 clientes (tipo TSP)
# ============================================================================
class RepartoClientes(ProblemaBusqueda):
    """Estado: (ubicación, clientes_visitados). Acción: ir a un cliente aún no visitado;
    cuando ya se visitaron todos, volver al depósito. Costo: distancia en km.
    Meta: estar en el depósito con todos los clientes visitados.
    (Distancias de ejemplo; se pueden cambiar por las reales.)"""
    nombre = "Reparto con 4 clientes"
    DEPOSITO = "Depósito"
    CLIENTES = ["A", "B", "C", "D"]
    DIST = {  # km, simétricas
        ("Depósito", "A"): 4, ("Depósito", "B"): 7, ("Depósito", "C"): 6, ("Depósito", "D"): 9,
        ("A", "B"): 3, ("A", "C"): 5, ("A", "D"): 8,
        ("B", "C"): 4, ("B", "D"): 5,
        ("C", "D"): 3,
    }

    def d(self, x, y):
        return self.DIST.get((x, y)) or self.DIST[(y, x)]

    def estado_inicial(self):
        return (self.DEPOSITO, frozenset([self.DEPOSITO]))

```

```
    def todos(self):
        return frozenset([self.DEPOSITO] + self.CLIENTES)

    def acciones(self, estado):
        ubic, vis = estado
        pendientes = [c for c in self.CLIENTES if c not in vis]
        if pendientes:
            return pendientes                    # ir al cliente X
        return [self.DEPOSITO] if ubic != self.DEPOSITO else []   # volver

    def resultado(self, estado, accion):
        return (accion, estado[1] | {accion})

    def costo(self, estado, accion, nuevo):
        return self.d(estado[0], accion)

    def es_meta(self, estado):
        return estado[0] == self.DEPOSITO and estado[1] == self.todos()


# ======================================================================
# 5. Análisis: solución óptima y métricas de complejidad
# ======================================================================
def analizar(problema, G):
    inicio = problema.estado_inicial()
    metas = [n for n in G if problema.es_meta(n)]
    camino = nx.dijkstra_path(G, inicio, metas[0], weight="weight")
    costo = nx.dijkstra_path_length(G, inicio, metas[0], weight="weight")
    # profundidad de la solución (nº de acciones) = camino más corto en pasos
    d = nx.shortest_path_length(G, inicio, metas[0])
    V, E = G.number_of_nodes(), G.number_of_edges()
    b = E / V                                    # factor de ramificación promedio
    bmax = max(dict(G.out_degree()).values())   # ramificación máxima (para la cota O(b^d))
    return {"camino": camino, "costo": costo, "d": d, "V": V, "E": E, "b": b, "bmax": bmax, "metas": metas}


def imprimir_resumen(problema, r):
    print(f"\n=== {problema.nombre} ===")
    print(f"Estados (nodos)        : {r['V']}")
    print(f"Transiciones (aristas) : {r['E']}")
    print(f"Factor de ramificación : promedio ≈ {r['b']:.2f}, máximo b = {r['bmax']}")
    print(f"Profundidad solución   : d = {r['d']} acciones")
    print(f"Costo óptimo (Dijkstra): {r['costo']}")
    print(f"Cota en árbol (sin detectar repetidos): O(b^d) = {r['bmax']}^{r['d']} = {r['bmax'] ** r['d']:,} nodos")
    print(f"Búsqueda en grafo (con conjunto de visitados): O(V + E) = {r['V'] + r['E']} pasos")


# ======================================================================
# 6. Dibujo del grafo
# ======================================================================
def dibujar(problema, G, pos, r, etiquetas, archivo, titulo, mostrar_pesos=None, tam=1100):
    fig, ax = plt.subplots(figsize=(11, 7))
    inicio = problema.estado_inicial()
    colores = []
    for n in G:
        if n == inicio:
            colores.append("#2E9E5B")        # inicio
        elif n in r["metas"]:
            colores.append("#D64545")        # meta
        elif n in r["camino"]:
            colores.append("#F2B134")        # en la solución
        else:
            colores.append("#9DB7E8")
    ruta = list(zip(r["camino"][:-1], r["camino"][1:]))
    nx.draw_networkx_edges(G, pos, ax=ax, edge_color="#B8BCC8", arrows=True, arrowsize=10,
                           node_size=tam, connectionstyle="arc3,rad=0.08", width=1)
    nx.draw_networkx_edges(G, pos, ax=ax, edgelist=ruta, edge_color="#E07B00", width=3,
                           arrows=True, arrowsize=16, node_size=tam, connectionstyle="arc3,rad=0.08")
    nx.draw_networkx_nodes(G, pos, ax=ax, node_color=colores, node_size=tam, edgecolors="#33384A")
    nx.draw_networkx_labels(G, pos, labels=etiquetas, ax=ax, font_size=8)
    if mostrar_pesos:
        nx.draw_networkx_edge_labels(G, pos, ax=ax, edge_labels=mostrar_pesos, font_size=8,
                                     font_color="#B00020", label_pos=0.3)
    ax.set_title(f"{titulo}\n{r['V']} estados · {r['E']} transiciones · costo óptimo = {r['costo']}",
                 fontsize=12, fontweight="bold")
    ax.text(0.01, 0.01, "Verde = inicio · Rojo = meta · Amarillo/naranja = solución óptima",
            transform=ax.transAxes, fontsize=8, color="#555")
    ax.axis("off")
    fig.tight_layout()
```

```
    fig.savefig(os.path.join(FIG, archivo), dpi=150)
    plt.close(fig)


def verificar_por_fuerza_bruta(p):
    """Prueba las n! rutas posibles del reparto y devuelve [(costo, ruta), ...] ordenada.
    Sirve para comprobar que Dijkstra sobre el grafo encontró realmente el óptimo."""
    rutas = []
    for perm in itertools.permutations(p.CLIENTES):
        ruta = [p.DEPOSITO, *perm, p.DEPOSITO]
        costo = sum(p.d(a, b) for a, b in zip(ruta, ruta[1:]))
        rutas.append((costo, ruta))
    return sorted(rutas)


def grafica_crecimiento():
    """Compara cuántas rutas hay (n!) con cuántos estados distintos tiene el grafo."""
    ns = list(range(2, 16))
    estados = [2 + n * 2 ** (n - 1) for n in ns]
    rutas = [math.factorial(n) for n in ns]
    fig, ax = plt.subplots(figsize=(8, 4.2))
    ax.semilogy(ns, rutas, "o-", color="#E8871E", label="Rutas posibles  n!")
    ax.semilogy(ns, estados, "s-", color="#0E4D64", label="Estados del grafo  2 + n·2^(n-1)")
    ax.set_xlabel("Número de clientes (n)")
    ax.set_ylabel("Cantidad (escala logarítmica)")
    ax.set_title("Crecimiento del espacio de búsqueda en el problema de reparto", fontsize=11, fontweight="bold")
    ax.grid(True, which="both", alpha=0.3)
    ax.legend()
    fig.tight_layout()
    fig.savefig(os.path.join(FIG, "crecimiento_estados.png"), dpi=150)
    plt.close(fig)


def main():
    # ---------- Problema 1: robot de almacén ----------
    p1 = RobotAlmacen()
    G1 = generar_grafo(p1)
    r1 = analizar(p1, G1)
    imprimir_resumen(p1, r1)
    print("Ruta óptima:", " -> ".join(str(e) for e in r1["camino"]))
    pos1 = {(f, c): (c, -f) for (f, c) in G1}                       # posición = celda del mapa
    et1 = {n: f"{n[0]},{n[1]}" for n in G1}
    pesos1 = {(u, v): w["weight"] for u, v, w in G1.edges(data=True) if w["weight"] > 1}
    dibujar(p1, G1, pos1, r1, et1, "grafo_almacen.png",
            "Problema 1 · Robot de almacén (rojo = costo 3 por zona de tráfico)", pesos1)

    # ---------- Problema 2: reparto con 4 clientes ----------
    p2 = RepartoClientes()
    G2 = generar_grafo(p2)
    r2 = analizar(p2, G2)
    imprimir_resumen(p2, r2)
    print("Ruta óptima:", " -> ".join(n[0] for n in r2["camino"]))
    for n in G2:   # capa = cuántos puntos se han visitado (para ordenar el dibujo)
        G2.nodes[n]["capa"] = len(n[1]) + (1 if n[0] == p2.DEPOSITO and len(n[1]) > 1 else 0)
    pos2 = nx.multipartite_layout(G2, subset_key="capa", align="vertical")
    et2 = {n: ("Dep" if n[0] == p2.DEPOSITO else n[0]) for n in G2}
    dibujar(p2, G2, pos2, r2, et2, "grafo_reparto.png",
            "Problema 2 · Reparto con 4 clientes (nodo = ubicación + clientes visitados)", tam=700)

    # ---------- Verificación por fuerza bruta ----------
    todas = verificar_por_fuerza_bruta(p2)
    print(f"Fuerza bruta ({len(todas)} rutas): mejor costo = {todas[0][0]} -> "
          f"coincide con Dijkstra: {todas[0][0] == r2['costo']}")

    # ---------- Complejidad teórica del problema 2 ----------
    n = len(p2.CLIENTES)
    print(f"\nReparto con n={n} clientes: estados = 2 + n·2^(n-1) = {2 + n * 2 ** (n - 1)}; "
          f"rutas posibles = n! = {math.factorial(n)}")
    grafica_crecimiento()
    print("Con n=10 clientes: n! = 3,628,800 rutas (explosión combinatoria) -> se requieren heurísticas (S3).")


if __name__ == "__main__":
    main()
```

B Anexo B. Salida de la ejecución

Resultado de ejecutar `python s2_problemas_busqueda.py`:

```
=== Robot de almacén ===
Estados (nodos)        : 21
Transiciones (aristas) : 50
Factor de ramificación : promedio ≈ 2.38, máximo b = 4
Profundidad solución   : d = 8 acciones
Costo óptimo (Dijkstra): 8
Cota en árbol (sin detectar repetidos): O(b^d) = 4^8 = 65,536 nodos
Búsqueda en grafo (con conjunto de visitados): O(V + E) = 71 pasos
Ruta óptima: (0, 0) -> (1, 0) -> (2, 0) -> (3, 0) -> (4, 0) -> (4, 1) -> (4, 2) -> (4, 3) -> (4, 4)

=== Reparto con 4 clientes ===
Estados (nodos)        : 34
Transiciones (aristas) : 56
Factor de ramificación : promedio ≈ 1.65, máximo b = 4
Profundidad solución   : d = 5 acciones
Costo óptimo (Dijkstra): 21
Cota en árbol (sin detectar repetidos): O(b^d) = 4^5 = 1,024 nodos
Búsqueda en grafo (con conjunto de visitados): O(V + E) = 90 pasos
Ruta óptima: Depósito -> A -> B -> D -> C -> Depósito
Fuerza bruta (24 rutas): mejor costo = 21 -> coincide con Dijkstra: True

Reparto con n=4 clientes: estados = 2 + n·2^(n-1) = 34; rutas posibles = n! = 24
Con n=10 clientes: n! = 3,628,800 rutas (explosión combinatoria) -> se requieren heurísticas (S3).
```